

Courses	Tutorials	Jobs	Practice	Upcoming Contests
Write an Article				
Write an Interview Experience				
Adding new column to existing DataFrame in Pandas				
Python map() function				
Read JSON file using Python				
How to get column names in Pandas dataframe				
Taking input in Python				
Read a file line by line in Python				
Python Dictionary				
Iterate over a list in Python				
Python program to convert a list to string				
Reading and Writing to text files in Python				
Enumerate() in Python				
Python String replace() Method				
Different ways to create Pandas Dataframe				
sum() function in Python				
Convert integer to string in Python				
Print lists in Python (5 Different Ways)				
*args and **kwargs in Python				
Python Program to convert String to a List				
Get a list as input from user in Python				

DSA	Data Structures	Algorithms	Interview Preparation	Data Science	Topic-wise Practice	C	C++	Java	JavaScript	Python	Latest Blogs	Competitive Programming	Machine Learning	Aptitude	Write & Earn	Web Development	Puzzles	Projects
applications																		
How to Install PIP on Windows ?																		
Python Lists																		
Create a Pandas DataFrame from Lists																		
Python String split()																		
Check if element exists in list in Python																		
How to drop one or multiple columns in Pandas Dataframe																		
floor() and ceil() function Python																		
Graph Plotting in Python Set 1																		
GET and POST requests using Python																		
Find average of a list in python																		

Adam Optimizer in Tensorflow

Last Updated : 09 Dec, 2022

Read	Discuss	Courses	Practice	Video
----------------------	-------------------------	-------------------------	--------------------------	-----------------------

Optimizers are algorithms or methods that are used to change or tune the attributes of a neural network such as layer weights, learning rate, etc. in order to reduce the loss and in turn improve the model. In this article, I am going to talk about Adam optimizer and its implementation in Tensorflow.

Before starting the discussion let's talk a little about [momentum](#) and [RMSprop](#).

Momentum Optimizer

The momentum optimizer is an extension of the standard [gradient descent algorithm](#). The normal gradient descent approach would need you to move more quickly in one direction while moving more slowly in the opposite direction, which would slow the algorithm down. Momentum limits oscillation to one direction, which speeds up the convergence of our method. This will also allow us to choose a higher learning rate because there is a limit on the number of steps that can be taken in the y-direction.

The following is the formula for momentum:

$$V_{dw} = \beta V_{dw} + (1 - \beta) g_t$$

$$w_t = w_{t-1} - \eta v_{dw}$$

where **w** is the weight, **beta** is the momentum factor, **g** is the gradient value and **eta** is the learning rate.

RMSprop Optimizer

The gradient descent algorithm with momentum and the RMSprop optimizer are comparable. The RMSprop optimizer limits oscillations that move vertically. As a result, we can speed up learning and our algorithm will be able to make bigger horizontal strides and converge more quickly. The method used to calculate the gradients differs between RMSprop and gradient descent. The calculations for the gradients for the RMSprop are shown in the following formulae.

$$v_{dw} = \beta v_{dw} + (1 - \beta) g_t^2$$

$$w_t = w_{t-1} - \eta \frac{g_t}{\sqrt{v_{dw} + \epsilon}}$$

where **w** is the weight, **beta** is the momentum factor, **g** is the gradient value and **eta** is the learning rate. **Epsilon** is a very small value that we use to avoid division by zero.

Now that we have an understanding of momentum and RMSprop optimization algorithms, let's take a closer look at how the Adam algorithm works.

Adam Optimizer

Adam (Adaptive Moment Estimation) is an adaptive optimization algorithm that was created specifically for [deep neural network](#) training. It can be viewed as a fusion of momentum-based stochastic gradient descent and RMSprop. It scales the learning rate using squared gradients, similar to RMSprop, and leverages momentum by using the gradient's moving average rather than the gradient itself, similar to [SGD](#) with momentum.

To estimate momentum, Adam uses exponential moving averages computed on the gradients evaluated on the current mini-batch. Mathematically, this can be written as :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where m and v are the moving averages and g is the gradient value. The betas are hyper-parameters whose good default values are, as suggested in the paper, 0.9 and 0.999 respectively.

Now as expectation values of the moments and gradient value should be equal to each other, we take the mean value of the moments, like:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Using all this information, Adam updates the weights using the following formula which is quite similar to the formula we use in RMSprop:

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

where w is the weight, eta is the learning rate and epsilon is an infinitely small value, usually 10⁻⁸, which we use to avoid division by zero.

Adam Optimizer in Tensorflow

You can pass string value adam to the optimizer argument of the model.compile functions like:

```
model.compile(optimizer="adam")
```

This method passes an adam optimizer object to the function with default values for betas and learning rate. You can use the Adam class provided in tf.keras.optimizers.

It has the following syntax:

```
Adam(learning_rate, beta_1, beta_2, epsilon, amsgrad, name)
```

The following is the description of the parameters given above:

- **learning_rate**: The learning rate to use in the algorithm. It defaults to a value of 0.001.
- **beta_1**: The value for the exponential decay rate for the 1st-moment estimates. It has a default value of 0.9.
- **beta_2**: The value for the exponential decay rate for the 1st-moment estimates. It has a default value of 0.999.
- **epsilon**: A small constant for numerical stability. It defaults to 1e-7.
- **amsgrad**: It is a boolean that specifies whether to apply the AMSGrad variant of this algorithm from the paper "On the Convergence of Adam and beyond". It has a default value of False.
- **name**: The Optional name for the operations created when applying gradients. Defaults to "Adam".

Let us go through an example in [Tensorflow](#) to better understand the usage of [Adam optimizer](#).

Python3

```
import tensorflow as tf
```

Now let's create the model. For this purpose, I am using a very simple [Neural Network](#) with 2 [Dense layers](#). The following piece of code defines the architecture of the model:

Python3

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#).

Got It !

Google এর মাধ্যমে GeeksforGeeks এ প্রবেশ করুন



Sandipan Dey
sandipan.dey.umbc@gmail.com



Sandipan Dey
geekforeverumbc@gmail.com



Start Your Coding Journey Now!

Login

Register

```
X = tf.keras.layers.Dense(10, 'relu')(X_input)
X_output = tf.keras.layers.Dense(2, 'softmax')(X)
model = tf.keras.Model(inputs=X_input, outputs=X_output)
return model
```

We can simply create the model now.

Python3

```
model = createModel((10, 10))
```

The model summary is as follows:

Python3

```
print(model.summary())
```

Output:

```
Model: "model_1"
Layer (type)                 Output Shape          Param #
-----
input_4 (InputLayer)        [(None, 10, 10)]      0
dense_5 (Dense)              (None, 10, 10)        110
dense_6 (Dense)              (None, 10, 2)         22
Total params: 132
Trainable params: 132
Non-trainable params: 0
```

Now, let's print out the weights of the model before training.

Python3

```
print('Initial Layer Weights')
print()
for i in range(1, len(model.layers)):
    print('Weight for Layer ' +str(i)+' : ')
    print(model.layers[i].get_weights()[0])
    print()
```

Output:

```
Initial Layer Weights

Weight for Layer 1:
[[-0.00454354 -0.44542956 -0.10721684 -0.36822182 -0.19939065 -0.13227496
 -0.17710215  0.04001284  0.10877353  0.30178553]
 [-0.5216929  -0.4426649  -0.36846456 -0.27782592  0.22852647 -0.46884304
  0.5094081  -0.5200859  -0.14566916 -0.53995645]
 [ 0.44872284 -0.17048255  0.0103073  -0.4445003  0.17243952 -0.3099317
  0.1728347  -0.19603652 -0.08578026 -0.42769736]
 [-0.2798235  -0.512177  0.4149052  -0.36846912  0.01805264  0.49091244
 -0.04532737  0.0092628  -0.09915763  0.17516673]
 [-0.2266348  0.11900622  0.22477901  0.14240289  0.3060978  -0.03541183
 -0.21709788 -0.31020045  0.3119203  -0.5285973 ]
 [-0.5345329  -0.17863265 -0.19109914 -0.10083333 -0.04073238  0.40429717
  0.15793598 -0.31843895 -0.1543011  0.44268316]
 [-0.00076026 -0.3441398  0.04974318  0.21111268  0.23431492  0.49153912
 -0.2852673  -0.01942509  0.12980881  0.24061912]
 [-0.41673392 -0.44042113  0.46301675  0.15985379  0.29592335 -0.37719238
  0.0019307  0.44035004  0.23492372 -0.13148385]
 [-0.33889228  0.15293491 -0.00629152 -0.5065441  -0.4919553  -0.44824737
  0.23605353 -0.05271649  0.36452303 -0.5051129 ]
 [-0.4087531  -0.33157796 -0.22369948 -0.28044367 -0.09715742 -0.05005264
  0.14658302  0.5138676  0.4340195  -0.16693294]]

Weight for Layer 2:
[[ 0.44999653  0.6937985 ]
 [-0.37577996 -0.30906732]
 [ 0.34833378 -0.11500055]
 [-0.42601848 -0.17034155]
 [-0.55422515  0.24619424]
 [-0.18209428 -0.04512733]
 [ 0.27548105  0.04039574]
 [ 0.3361835  -0.35830352]
 [-0.06059337 -0.26052788]
 [-0.30983666 -0.1068266 ]]
```

Let's get some dummy data to pass on to the model.

Python3

```
tf.random.set_seed(5)
X = tf.random.normal((2, 10, 10))
Y = tf.random.normal((2, 10, 2))
```

Now it's time to compile the model. I have used the 'adam' optimizer, 'categorical_crossentropy' loss, and 'accuracy' metrics.

Python3

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

The optimizer has the following configurations.

Python3

```
print(model.optimizer.get_config())
```

Output:

```
{'name': 'Adam',
 'learning_rate': 0.001,
 'decay': 0.0, 'beta_1': 0.9,
 'beta_2': 0.999,
 'epsilon': 1e-07,
 'amsgrad': False}
```

Now let's fit the dataset to the model.

Python3

```
model.fit(X,Y)
```

Start Your Coding Journey Now!

The model has now been trained. Let’s check the weights after training.

Python3

```
print('Final Layer Weights')
print()
for i in range(1, len(model.layers)):
    print('Weight for Layer '+str(i)+' : ')
    print(model.layers[i].get_weights()[0])
    print()
```

Output:

```
Final Layer Weights

Weight for Layer 1:
[[-0.00654349 -0.44342965 -0.10921675 -0.3662219  -0.1973907  -0.1302751
 -0.17510226  0.03801289  0.1107733   0.30378544]
 [-0.52369285 -0.44066504 -0.37046447 -0.275826   0.23052633 -0.46684325
  0.50740826 -0.52208585 -0.14766905 -0.5379566 ]
 [ 0.44672292 -0.17248243  0.00838751 -0.44650015  0.17443928 -0.30793315
  0.17483088  0.19403683 -0.0077801  -0.4296968 ]
 [-0.27782366 -0.51017714  0.41290528  0.3664692  0.02005251  0.4929123
 -0.04332761  0.00726285 -0.10115752  0.17716667]
 [-0.22463483  0.11700632  0.22677837  0.14040303  0.30809772 -0.03741121
 -0.21509793 -0.31219885  0.3139191  -0.53059703]
 [-0.53653216 -0.17663273 -0.18909931 -0.09883394 -0.04273209  0.40629518
  0.15993057 -0.31643903 -0.15230125  0.44068348]
 [ 0.00123966 -0.3461398  0.04774461 -0.21311262  0.23631434  0.48953933
 -0.28326756 -0.02142486 -0.13180864  0.23861921]
 [-0.41473395 -0.442421  0.46101683  0.16185364  0.29792327 -0.37919164
  0.07993093  0.43835813  0.23292506 -0.13348362]
 [-0.33689255  0.15493423 -0.08829022 -0.50054397 -0.49395528 -0.45024717
  0.23405361 -0.05071672  0.36252397 -0.50711274]
 [-0.410753   -0.32957822 -0.22169971 -0.28244346 -0.09915712 -0.05205179
  0.1445832   0.5158675   0.43601942 -0.16893177]]

Weight for Layer 2:
[[ 0.44799674  0.6918039 ]
 [ -0.37377998 -0.31186724]
 [ 0.3463338  -0.11300066]
 [-0.42401856 -0.1723415 ]
 [-0.5562251   0.24419433]
 [-0.18409316 -0.04712712]
 [ 0.27348107  0.03839577]
 [ 0.3341835  -0.3603035 ]
 [-0.06259333 -0.26252785]
 [-0.31183657 -0.10882653]]
```

Related Articles

- 1. Numpy Gradient – Descent Optimizer of Neural Networks
- 2. Why TensorFlow is So Popular – Tensorflow Features
- 3. Python | Classify Handwritten Digits with Tensorflow
- 4. Introduction to TensorFlow
- 5. Introduction to Tensor with Tensorflow
- 6. Python | Tensorflow cos() method
- 7. Python | Tensorflow nn.sigmoid()
- 8. Python | Tensorflow nn.relu() and nn.leaky_relu()
- 9. Python | Tensorflow nn.softplus()
- 10. Python | Tensorflow nn.tanh()

Like 0

Previous

How to Add Text to Merged Cell using XLSX Package – R

Next

Power BI – How to Create a Treemap?

Article Contributed By :



aayushmohansinha

@aayushmohansinha

Vote for difficulty

- Easy
- Normal
- Medium
- Hard
- Expert

Article Tags : Picked, Technical Scripter 2022, Python, Technical Scripter

Practice Tags : python

- Improve Article
- Report Issue

GeeksforGeeks

A-143, 9th Floor, Sovereign Corporate Tower,
Sector-136, Noida, Uttar Pradesh - 201305
feedback@geeksforgeeks.org

Company

- About Us
- Careers
- In Media
- Contact Us
- Privacy Policy
- Copyright Policy
- Advertise with us

Computer Science

- Operating Systems
- Computer Network
- Database Management System
- Software Engineering
- Digital Logic Design
- Computer Graphics
- Engineering Maths

Languages

- Python
- Java
- C++
- GoLang
- SQL
- R Language
- Android Tutorial

Data Science & ML

- Data Science With Python
- Data Science For Beginner
- Machine Learning Tutorial
- Maths For Machine Learning
- Pandas Tutorial
- NumPy Tutorial
- OpenCV Python Tutorial

Data Structures

- Array Data Structure
- String Data Structure
- Linked List Data Structure
- Stack Data Structure
- Queue Data Structure
- Tree
- Graph

Interview Corner

- Company Preparation
- Preparation for SDE
- Company Interview Corner
- Experienced Interview
- Internship Interview
- Competitive Programming

Algorithms

- Sorting
- Searching
- Greedy
- Dynamic Programming
- Pattern Searching
- Recursion
- Backtracking

Python

- Python Tutorial
- Python Programming Examples
- Django Tutorial
- Python Projects
- Python Tkinter
- OpenCV Python Tutorial

Web Development

- HTML
- CSS
- JavaScript
- Bootstrap
- ReactJS
- AngularJS
- NodeJS

School [Class 6-12]

- CBSE Notes for Class 8
- CBSE Notes for Class 9
- CBSE Notes for Class 10
- CBSE Notes for Class 11
- CBSE Notes for Class 12
- English Grammar

Write & Earn

- Write an Article
- Improve an Article
- Pick Topics to Write
- Write Interview Experience
- Internships
- Video Internship

UPSC/SSC/BANKING

- SSC CGL Syllabus
- SBI PO Syllabus
- IBPS PO Syllabus
- UPSC Ethics Notes
- UPSC Economics Notes
- UPSC History Notes

Start Your Coding Journey Now!